



جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Sprint 2

Logistics Paperwork Process Automation System (Web-Based Document Upload, Data Extraction, and Form Generation)

Course

BIS405 – Graduation Project

Area

Digital Systems and Solution Design

Team Name:

Team Members & Tasks:

Norah Alshehri (2230004682): Backend & Extraction

Bayader Alghamdi (2230003715): Backend & Extraction

Shaykhah Mohsen (2230005764): Frontend & UI/UX

Maha Alnafea (2230003005): Frontend & UI/UX

Supervisor: Dr. Linah

Date: 4/2/2026

In Sprint 2, our team successfully developed and integrated the core data extraction engine and the interactive review interface, marking a major milestone in our Logistics Paperwork Process Automation System. We engineered a robust backend utilizing the `pdfplumber` library and Regular Expressions to automatically parse unstructured PDF commercial invoices and accurately extract critical business fields. Simultaneously, we built a modern, responsive React.js frontend featuring a dual-pane layout. This intuitive side-by-side interface allows operations staff to view the original document while seamlessly verifying and approving the extracted data before securely saving it to the database.

Status: Data Approved and Saved Successfully!

Sprint Goal

2 | Page

Accomplishments

The team successfully delivered a fully functional End-to-End (E2E) pipeline. A user can now upload a commercial invoice (PDF), have the system automatically read and extract key data points, review the document and data side-by-side, and save the validated data securely to the database.

The work was divided into two main streams to ensure parallel development and efficiency:

A. Backend & Data Extraction (Assigned to: Noura & Bayader)

API Development (FastAPI):

Developed robust RESTful API endpoints for managing cases (`POST /cases/`), uploading documents (`POST /cases/{case_id}/documents/`), and handling final approvals (`PUT /documents/{document_id}/approve`).

Database Integration (SQLite & SQLAlchemy):

Designed and migrated relational database models to store system users, cases, uploaded documents, and the granular ExtractedFields (storing both raw AI/script extracted values and human-approved values).

Automated Extraction Algorithm:

Implemented `pdfplumber` to accurately parse text from PDF documents.

- Developed a custom Pattern-Matching (Regex) algorithm tailored for English commercial invoices.
- Successfully automated the extraction of four critical fields: Vendor Name, Invoice Number, Ship-ment Date, and Total Amount.

B. Frontend & User Interface (Assigned to: Sheikha & Maha)

Environment Setup:

Initialized a modern, high-performance web application using React.js (via Vite).

Side-by-Side Review UI:

Designed and developed the core interactive workspace. The interface features a dual-pane layout:

- *Left Pane:* An embedded, real-time PDF viewer for the uploaded document.
- *Right Pane:* A dynamic, auto-populating form displaying the extracted data for human review.

API Integration (Axios):

Engineered seamless asynchronous communication with the backend. Implemented multipart/form-data handling for PDF uploads and JSON parsing for retrieving extraction results.

Modern UI/UX Styling:

Applied advanced CSS techniques (Soft UI, subtle box-shadows, responsive flexbox layouts, and modern typography) to deliver a polished, commercial-grade user experience.

Data Validation & Approval:

Implemented the "Approve & Save" workflow, allowing users to correct any extraction inaccuracies and push the finalized data to the backend database.

Technical Challenges & Resolutions

During the sprint, the team encountered and successfully resolved several technical challenges:

Language Directionality Complexities (RTL vs. LTR):

- *Issue:* Initial extraction attempts on Arabic invoices yielded inconsistent results due to Right-to-Left (RTL) text rendering issues within the PDF parsing library.
- *Resolution:* The team pivoted to standardizing the system on English invoices. This aligns with the academic requirements of the graduation project and significantly stabilized the Regex pattern-matching accuracy.
- *Note:* *We will improve extraction to work with Arabic invoices in next stages.*

Cross-Origin Resource Sharing (CORS) Restrictions:

- *Issue:* The React frontend (Port 5173) was blocked from communicating with the FastAPI backend (Port 8000) by the browser's security policies.
- *Resolution:* Implemented and configured CORSMiddleware in the FastAPI application to explicitly allow cross-origin requests, enabling seamless local development¹.

The issue was caused by the browser's same-origin policy, because the React frontend on localhost:5173 and the FastAPI backend on localhost:8000 are considered different origins due to the different ports. According to the FastAPI documentation, the proper solution is to configure CORSMiddleware and explicitly allow the frontend origin so the browser accepts cross-origin requests during development².

State Synchronization:

- *Issue:* Ensuring the frontend accurately tracked the database-generated document_id immediately after a file upload.
- *Resolution:* Refactored the backend upload endpoint to return the newly created document_id in its JSON response, which the React state manager then utilizes for subsequent extraction and approval requests.

```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS A:\Gra Project> uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['A:\\Gra Project']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15584] using StatReload
ERROR: Error loading ASGI app. Could not import module "main".
```

[1] developer.mozilla.org

[2] [fastapi](https://fastapi.tiangolo.com/)

```
Download the React DevTools for a better development experience: https://react.dev/link/react-devtools react-dom client.js?v=87611248:14338
❌ ▶ AxiosError: Network Error at async handleFileUpload (App.jsx:31:25) App.jsx:42
    Show ignore-listed frames
❌ Failed to load resource: net::ERR_CONNECTION_REFUSED 127.0.0.1:8000/cases/1/documents/:1
> ctrl i to turn on code suggestions. Don't show again
```

Conclusion & Next Steps

Sprint 2 was highly successful, culminating in a working prototype of the system's most complex feature: the extraction and review cycle. The foundational architecture for both the backend and frontend is now solid and proven.

Sprint 3 Planning:

With the core engine functional, the next sprint will focus on wrapping the system in a complete application shell. Planned tasks include:

- Enhance backend to support scanned invoices and arabic language.
- Developing the User Authentication UI (Login/Register screens).
- Building the main Administrator Dashboard (statistics and case overviews).
- Implementing the Case Management UI (creating and listing cases).
- Connecting all interfaces via React Router.

References

FastAPI (2026) *CORS (Cross-Origin Resource Sharing)*. Available at: <https://fastapi.tiangolo.com/tutorial/cors/>

MDN Web Docs (2026) *Cross-Origin Resource Sharing (CORS)*. Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS>